



Form4
Capstone Design Implementation
Design and Development of a WhatsApp-
Based Issue Reporting and Analytics
Dashboard System for TRAMOS Web at PT
Andakara Informatika Nusantara

GROUP MEMBER:

No.	Student Name	Student ID	Study Program
1.	Cayla Winri Azzahra	012202300098	Information System
2.	Ivander Johana Pratama	012202300030	Information System

Advisor : Hadi Suprayitno, S.Kom., M.M.

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

A.	DESIGNS IMPLEMENTATION	5
1.	Functions/Procedure/Class implementation.....	5
2.	Database implementation.....	7
3.	User Interface implementation.....	9
4.	Integration Module.....	12
B.	PRODUCT DISPLAY	14
C.	COMPONENT COST ANALYSIS.....	24
D.	FUNCTIONAL TESTING	25
E.	MANUAL GUIDE.....	28

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

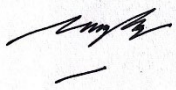
Name : 1. Cayla Winri Azzahra – 012202300098
2. Ivander Johana Pratama - 012202300030

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**Design and Development of a WhatsApp-Based Issue Reporting and Analytics Dashboard System for TRAMOS Web at PT Andakara Informatika Nusantara**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, 29 Mei 2026

Signer 1	Signer 2
	
Cayla Winri Azzahra - 012202300098	Ivander Johana Pratama - 012202300030

A. DESIGNS IMPLEMENTATION

1. Functions/Procedure/Class implementation

The implementation of the TRAMOS system includes three main modules: AI Chatbot Integration, Ticketing System Integration, and Analytics Dashboard. These three modules enable real-time reporting of issues, automatic ticket generation and analytical data processing based on requirements presented in Form 3.

1.1 WhatsApp Webhook and AI Intent Processing

This module is implemented by creating a Webhook Handler which integrates TRAMOS system with the Meta WhatsApp Business API. It serves as the major interface with the users which processes their incoming messages and initiates the required actions.

The key function `handle_incoming_message` accepts incoming message and parses its content before sending it to the AI processing flow.

```
@router.post("/webhook/whatsapp")
async def handle_incoming_message(request: Request) -> Response:
    body = await request.json()
    entry = body.get("entry", [{}])[0]
    changes = entry.get("changes", [{}])[0]
    value = changes.get("value", {})

    messages = value.get("messages", [])
    if not messages:
        return Response(content=json.dumps({"status": "ok"}), media_type="application/json")

    message = messages[0]
    from_phone = message.get("from")
    message_body = message.get("text", {}).get("body", "").strip()

    response_text = await process_message_with_ai(
        message_body=message_body,
        user_name="User",
        phone_number=from_phone
    )
```

`process_message_with_ai` function provides implementation of a conversation-based AI system using state machine technique. It enables interaction with the user in various states such as greeting, problem collecting and confirmation.

```
async def process_message_with_ai(message_body: str, user_name: str, phone_number: str) -> str:
    session = session_manager.get_or_create_session(phone_number)

    if session.current_state == DialogState.GREETING:
        response, next_state = DialogFlowHandler.handle_greeting(session)

    elif session.current_state == DialogState.COLLECTING_PROBLEM:
        analysis = SmartDialogFlowHandler.analyze_problem(message_body)
        session.problem_category = analysis.get('category')
```

This solution relies upon Natural Language Processing (NLP) technique for analyzing intents of the sender and categorizing them accordingly. Such an approach to implementing AI helpdesk chatbot was described in previous works .

1.2 Auto-Ticketing API Integration

The role of the Ticketing System module is to deal with unresolvable issues through an interface with the external osTicket software using the REST API.

The function `_create_ticket_from_session` will be performed in the case when the issue is found out to be unresolvable. The chat session will be transformed into a ticket and will be sent to osTicket via the REST API.

```
async def _create_ticket_from_session(session, phone_number: str, user_name: str) -> str:
    subject = f"[TRAMOS] {session.problem_severity.upper()} - {session.problem_category}"

    ticket_request = CreateTicketRequest(
        name=user_name,
        email=f"{phone_number}@whatsapp.tramos.id",
        subject=subject,
        message=session.problem_description,
        source="whatsapp",
        ip="127.0.0.1"
    )

    result = await osticket_service.create_ticket(ticket_request)
```

Moreover, a specific API endpoint will be developed to facilitate this process:

```
@router.post("", response_model=CreateTicketResponse)
async def create_ticket(ticket_data: CreateTicketRequest) -> CreateTicketResponse:
    result = await osticket_service.create_ticket(ticket_data)

    if not result.success:
        raise HTTPException(status_code=500, detail=result.message)

    return result
```

This module ensures that every critical issue is recorded with a unique ticket ID and can be tracked by support operators. The integration follows best practices in Service-Oriented Architecture (SOA) by decoupling the chatbot system from the ticketing service .

1.3 Analytics Dashboard

The Analytics Dashboard module is implemented as a React-based frontend component to visualize data from the system and Machine Learning insights in real time.

`fetchData` function fetches data from the backend API, where the aggregated data include alerts, system health metrics, dashboard metrics, and ML insights.

```
const fetchData = async () => {
  try {
    setLoading(true);
    const [alertsData, healthData, dashData, mlData, logData] = await Promise.all([
      analyticsService.getActiveAlerts().catch(() => null),
      analyticsService.getHealthCheck().catch(() => null),
      analyticsService.getDashboard(dateRange.startDate, dateRange.endDate).catch(() => null),
      analyticsService.getMLInsights(dateRange.startDate, dateRange.endDate).catch(() => null),
      analyticsService.getActivityLog(15).catch(() => null),
    ]);
    setAlerts(alertsData);
    setHealth(healthData);
    setDashboard(dashData);
    setMLInsights(mlData);
    setActivityLog(logData);
  } catch (err) {
    console.error('Failed to load insights data');
  } finally {
    setLoading(false);
  }
};
```

The dashboard displays important metrics like total conversations, total tickets, and the efficiency of the AI in resolving tickets:

```
<div className="ml-stat">
  <span>Efektivitas AI Chatbot</span>
  <span>{kbEffectiveness.ai_resolution_rate || 0}%</span>
  <div>Diselesaikan AI: {kbEffectiveness.resolved_by_ai ||
    0} • Dieskalasi ke Tiket: {kbEffectiveness.escalated || 0}</div>
</div>
```

The purpose of this module is to enable management analysts to track system performance and analyze the trends with the help of real-time analytics.

2. Database implementation

In our project, we decide to use PostgreSQL as the main database management system to back the TRAMOS Issue Reporting System. The reason for this decision includes the efficiency of the PostgreSQL database for transactional data and relational data, as well as its reliability in maintaining ACID properties.

Database implementation will follow the structure specified in Form 3 according to the hierarchical and modular nature of the AI chatbot, ticketing system, and analytics dashboard.

Advantages of Utilizing PostgreSQL:

1. High Reliability (ACID Compliance)
Provides consistency in operations like creating tickets, updates, and AI processing.
2. Data Relationship Integrity
Foreign key relationships, for example, `conversation_id`, `ticket_id`, and `user_id`, guarantee proper relationships between tables.
3. Effective Query Execution
PostgreSQL supports joins and aggregate functions that are necessary for building analytic dashboards.
4. Scalability
Indexing and schema design allow the database to scale with growing numbers of conversations and tickets.



Figure 1 Database PostgreSQL

TRAMOS Database Architecture & Design Strategy:

The TRAMOS database is made up of several tables that represent various modules of the system, including Conversation & Messaging, AI Processing, Ticketing, Analytics Dashboard, and User & Session Management, where all entities are connected using foreign key relationships.

1. Conversation & Messaging System

The four tables: `conversations`, `message_history`, `conversation_turns`, and `conversation_context` have a one-to-many relationship design to facilitate chat history management, AI processing, and real-time tracking.

2. AI Processing & Solution Tracking

The three tables of: `solution_attempts`, `solutions`, and `solution_effectiveness` are used to track the performance of each attempt by recording every single AI generated response.

3. Ticketing System (osTicket Integration)

The table `tickets` is directly related to both `user_id` and `conversation_id` to allow for easy tracing from the point of interaction with the user to resolution.

4. Analytics & Dashboard System

The three tables of `analytics_data`, `dashboard_analytics_summary`, and `dashboard_users` are divided into raw and summary forms for optimization in query performance.

5. User & Session Management

The three tables: `users`, `user_profile_data`, and `whatsapp_sessions` are used for managing multi-roles, tracking sessions, and behavioral analytics.

3. User Interface implementation

TRAMOS' user interface implementation has been developed to ensure that it provides a contemporary and interactive interface for the entire user base, which includes drivers, staff members, support operators, and management analysts. Rather than adopting static prototyping techniques like Figma, the user interface was directly coded in a frontend application to ensure that all UI elements could instantly communicate with the backend services, APIs, and database modules.

The development of the user interface involved prioritizing the design elements of usability, accessibility, and efficiency, particularly for reporting and monitoring of any issues in real time. For instance, the frontend interface can easily facilitate interactions between the WhatsApp AI chatbot, ticket management system, and analytics dashboard. All UI elements have been programmed to be linked directly with backend services using API integrations, thus providing an efficient interface for receiving real-time updates, submitting reports, monitoring tickets, and gaining insights from the analytics dashboard. These UIs have been iteratively developed based on system requirements and UML diagrams described in Form 3.

1. Login Interface

The log-in interface forms the main entry point of authentication to access the TRAMOS portal. This design incorporates features such as an elegant modern user interface that helps increase user experience while retaining the professional enterprise look appropriate for logistics and transportation.

The design has provisions for authenticating users via the standard username-password scheme as well as supporting Google authentication for easy log-in into the system. Provision is made in the design for session management to ensure secure authentication and role-based access to users in the system.

From the implementation perspective, the log-in interface interfaces directly with the authentication services offered on the back end before users are allowed access to the dashboard system.

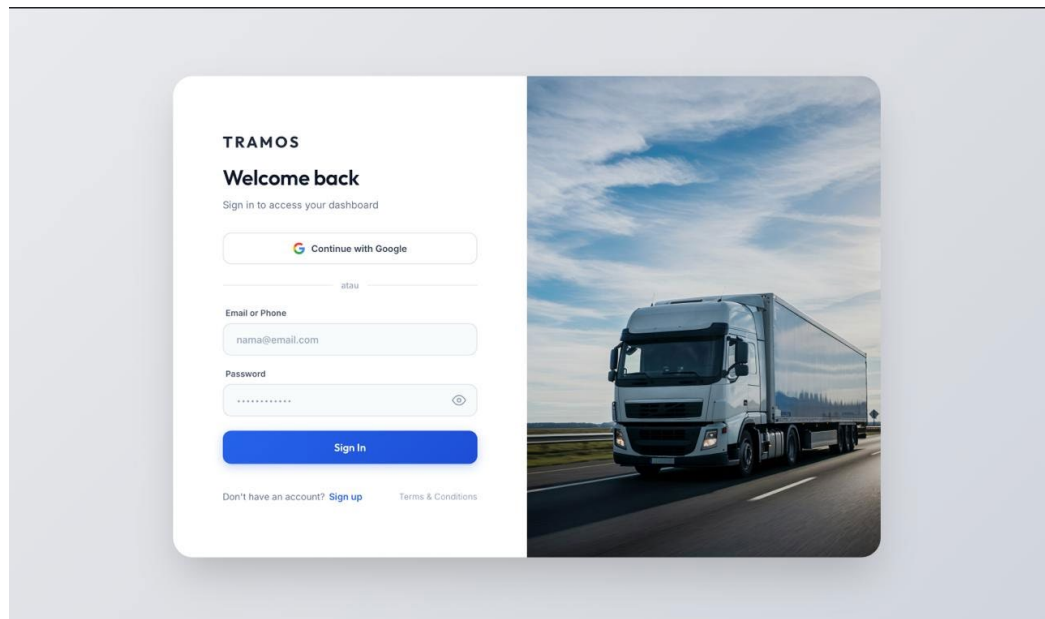


Figure 2 Login Page Interface

2. Dashboard Interface

The implementation of the Dashboard Interface is achieved through the integration of the monitoring and analytics feature within the TRAMOS platform. The Dashboard Interface will enable visualization of the real-time generated data from conversations on WhatsApp, analysis results, generation of the tickets, and system performance analytics.

The interface connects with the analytics module directly from the PostgreSQL database for summarization purposes. Some essential operational measures include total conversations, the rate of success in AI analysis, number of generated tickets, total messages, categories of issues, urgency of the issues, and quality of sessions. This implementation enables management analysts and support staff to monitor service condition and trends effectively and in real-time.

The interface uses modular dashboard components that include summary cards, statistics charts, and tickets distribution graphs. There are interactive filters included in the interface to enable users to filter data based on time frames, whether daily, weekly, or monthly. Such visualization components make it easier for management personnel to analyze trends in issue occurrence and system performances.

Furthermore, the interface enables the continuous synchronization of backend services. This means that any newly generated conversation data, tickets, and analyses are available for visualization automatically, without the need to manually refresh pages.

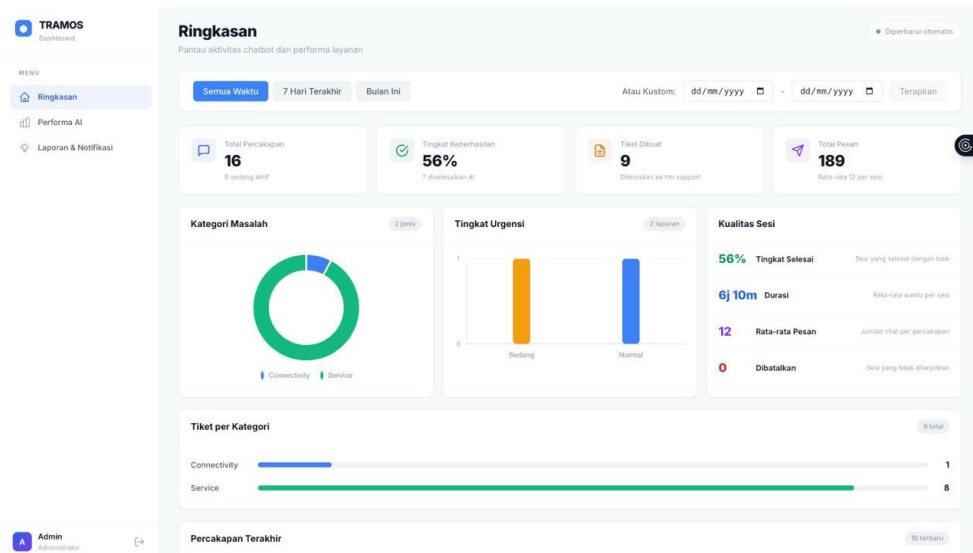


Figure 3 Dashboard Interface

3. Ticket Monitoring Interface (osTicket Integration)

The Ticket Monitoring Interface is implemented using the osTicket platform, which functions as the centralized helpdesk and ticket management system within TRAMOS. This interface is responsible for monitoring, managing, and updating all support tickets generated automatically from unresolved WhatsApp conversations processed by the AI chatbot system.

The integration between TRAMOS and osTicket will involve the use of REST API communication where tickets are generated automatically when the AI engine fails to resolve the user issue. Generated tickets are expected to include Important details such as the issue category, ticket subject, priority level, user information, and conversation reference. Such an implementation will allow for proper documentation of all the escalation processes.

Operationally, the interface will allow support operators to monitor tickets received, ticket status, handling priority, and updates on ticket resolution. The dashboard also enables ticket categorization and filter functionality which is expected to facilitate identification of high-priority logistics issues.

All ticket activities will be synchronized with the TRAMOS analytics database to enable ticket monitoring via the dashboard and machine learning algorithm analysis. As such, ticket data in osTicket can feed into issue trend analysis, operational reporting, and service performance monitoring..

osTicket greatly enhances the operational workflow management process through improved reporting and issue handling within a centralized.

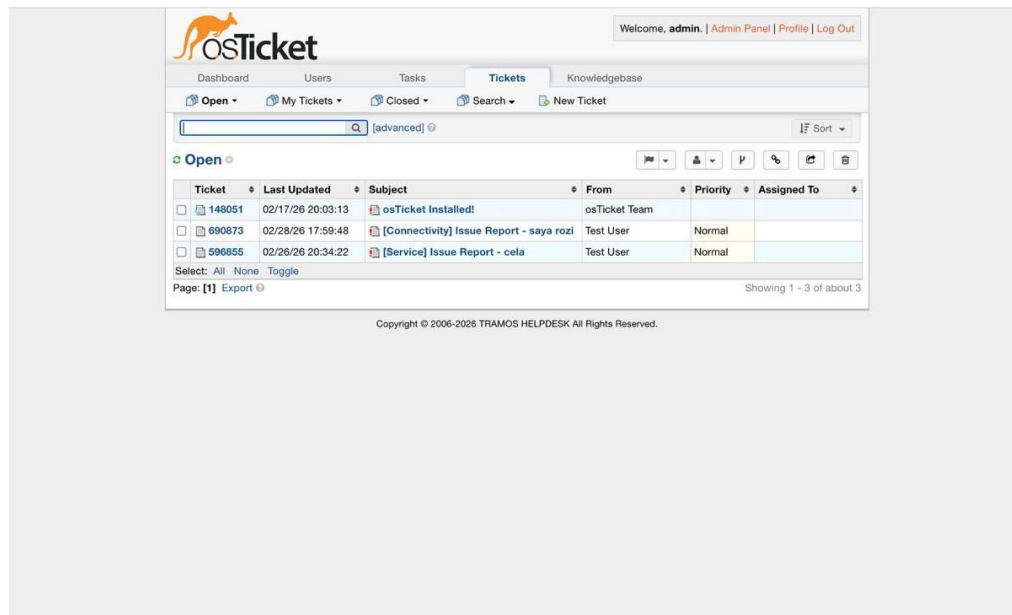


Figure 4 osTicket Dashboard Interface

4. Integration Module

TRAMOS utilizes a modular service-oriented architecture to facilitate smooth integration between the AI chatbot system, ticket management, analytics dashboard, and database services. Each module operates independently yet interconnected via REST API communication, shared databases, and data synchronization. Thus, the architecture allows for maintaining the scalability, maintainability, and efficiency of operations in all logistics support services.

The integration process starts from the WhatsApp Gateway module, where the users like drivers or employees create issue reports via messages on WhatsApp. The received messages are handled by the backend API service, and then are sent for further analysis and processing by the AI Processing Engine for intents recognition and issue classification. The AI engine performs the analysis of the user conversation using machine learning and natural language processing and identifies whether the problem is solvable by itself or needs to be escalated as a support ticket.

The results of the AI processing are saved in the PostgreSQL centralized database. The database serves as the main integration module that connects all other modules, including conversation, ticket, analytics, user session, and AI solution modules. Using such centralization approach, all modules get access to synchronized operational data.

In case the problem cannot be solved automatically, the integration between the backend and osTicket platforms takes place through REST API communication. The system automatically creates a ticket that includes issue categories, ticket subjects, ticket priority, and information about users. The integration guarantees the automatic escalation of any unsolved issues into helpdesk tickets without creating them manually. After that, the support operator can work on the ticket and change its status.

Also, the Analytics Dashboard module is integrated with the database and the ticketing system. Every conversation, AI response, ticket creation activity, and ticket resolution process is continually synchronized into the analytics engine. Then, the dashboard gets this operational data and uses it to create visualizations related to issue categories, urgency, AI success rate, total conversations, and tickets. Moreover, machine learning aggregation processes are performed to identify operational trends and give actionable insights to management analysts.

To keep a stable connection between modules, JSON API requests and database synchronization technologies are used. Besides, the integration between operational modules provides the possibility of system scalability in the future. For example, in case you need more services such as predictive maintenance analytics or fleet monitoring systems, you can integrate them easily.

In summary, the TRAMOS integration architecture guarantees that all of its elements, such as WhatsApp communication, AI, ticket escalation, database management, and analytics visualization, work together as one real-time intelligent logistics support system.

B. PRODUCT DISPLAY

1. SOFTWARE PRODUCT DISPLAY

The Overview Dashboard represents the key interface for monitoring the TRAMOS operational system. In general, this interface is supposed to show real-time data on activities related to the chatbot, tickets escalations, issue categories, and operational performance. The Overview Dashboard uses the analytics API and PostgreSQL database to show summarized information from WhatsApp conversations and ticketing processes.

First of all, the user will be able to access this interface once the authentication process is completed successfully. In general, this interface is used to see several operational metrics, including total conversations, AI resolution rate, number of created tickets, total messages, issue category distribution, urgency level distribution, and session quality.

Furthermore, this interface can be automatically refreshed and filtered based on specific date ranges. As far as visualization goes, the implementation is provided via interactive charts and summary cards that will allow management-level employees to analyze frequently mentioned issues without manual review of conversation records.

1. Authentication Interface

The login interface acts as the entry point of the TRAMOS dashboard system. The login interface acts as a secure access control system that ensures the protection of administrator, analyst, and operational access to the analytics dashboard and monitoring system. The authentication process has been implemented by integrating the interface with the backend authentication API, JWT token generation, and Google OAuth login services.

At the initial point of access to the TRAMOS dashboard system, the system presents the login page to the users. The design of the login interface is based on a modern and minimalistic approach. For users that already have accounts on the system, they can directly input their email or phone number, together with the password, to access the dashboard system. The system verifies the entered credentials by means of the FastAPI backend, and an access token is generated.

The login interface also supports Google Single Sign-On (SSO). By using this feature, users will be able to log in to the system by logging into their Google accounts.

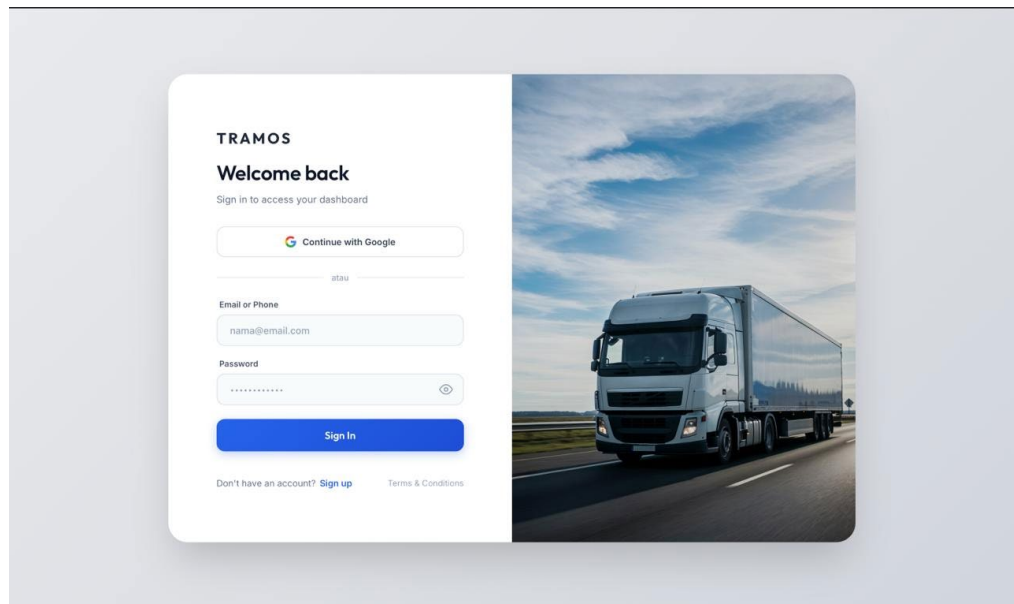


Figure 5 Login Interface

If a user does not have an account at all, then the Sign Up button is used to register the user by redirecting them through the registration process.

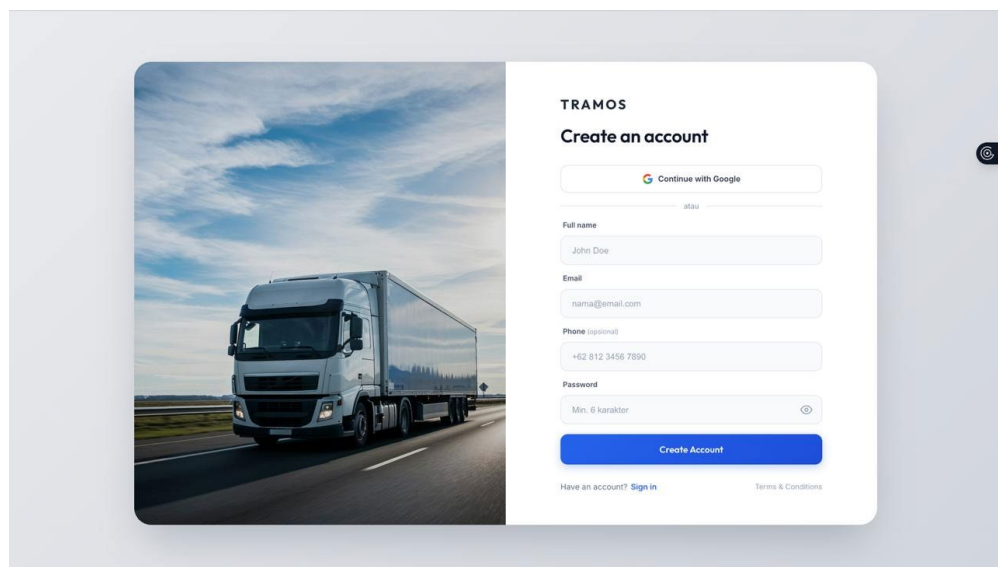


Figure 6 Signup Interface

While going through the registration procedure, users are required to fill in the account details including their name, email ID, phone number, and a password. Once the registration process is completed, the OTP verification code is generated and sent to the registered email ID. In order for the account to be considered verified, users are required to enter the correct OTP verification code.

This process is done to ensure that all authenticated users only access the dashboard features. Validation processes include duplicate emails, invalid entry, expired OTP codes, and failed verification. Apart from registering

manually, there is another way to register in the system by creating and accessing accounts using Google sign-in.

Upon verification, users can then directly go ahead to login into the dashboard system depending on the role assigned to them.

2. Overview Dashboard Interface

The user is shown the dashboard after successfully going through the authentication phase. The dashboard shows different operational metrics like total conversations, percentage of conversations resolved by AI, tickets created, total messages exchanged, distribution of issue categories, distribution of urgency levels, and session quality metrics.

These visualization tools have been designed in such a way that they show interactive graphs and summary cards for easy identification of trends and frequent issues. This interface makes it easier for management users to analyze the performance of the system without analyzing conversation logs individually.

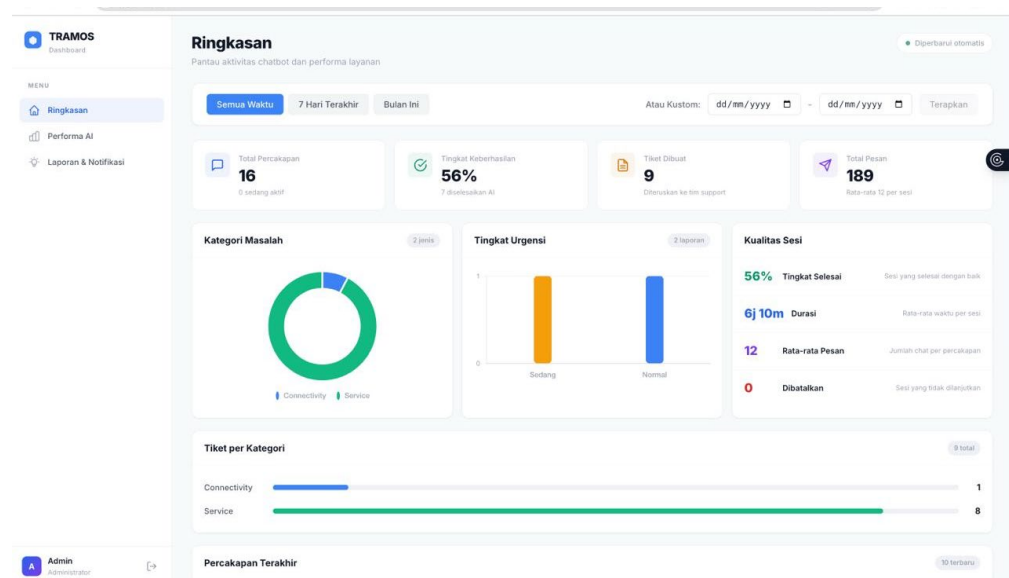


Figure 7 Dashboard Page Interface

Right next to the AI Performance Page is meant for assessing the effectiveness of the TRAMOS AI chatbot in dealing with reported issues and problems. The performance page will focus on chatbot performance analytics, which include successful AI solutions, escalations, abandoned conversations, and issue severities.

The page is designed to provide a performance score based on the conversation results and AI resolution. The system analyzes the percentage of issues that have been resolved by the chatbot without escalation into tickets. Other metrics like number of messages per conversation and number of escalations are also included in the performance score.

There are analytical visualizations and recommendations by the AI system to aid in improving Knowledge Base quality and troubleshooting workflows. This way, the administrators will be able to assess whether the chatbot is able to reduce the workload in manual support processes.

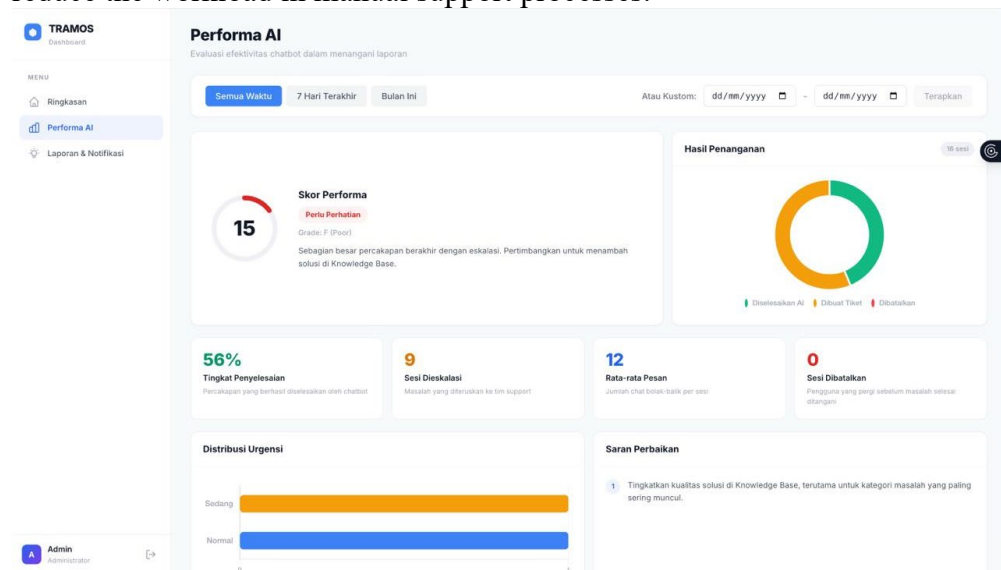


Figure 8 AI Performance Page

Another page is The Reports and Notifications, which serves as a centralized monitoring tool for analyzing system health, generating reports, and AI-based recommendations. This centralized monitoring interface gathers information from several modules such as database services, AI processing services, WhatsApp integration, osTicket integration, and analytics processing.

The dashboard shows various status indicators to check if major services are running well. In addition to these features, the interface generates weekly operational analysis and issue escalation trends to enable administrators to assess the performance of the entire service. Furthermore, the Reports and Notifications Dashboard presents activity logs and notifications to assist administrators in monitoring current operational events in real time. This tool helps monitor the system proactively and respond quickly to any recurring problems.

Moreover, the dashboard presents AI-based recommendations and operational summaries based on analytics data. The main aim of these recommendations is to facilitate managers in identifying frequently escalated issue categories and monitor AI performance.

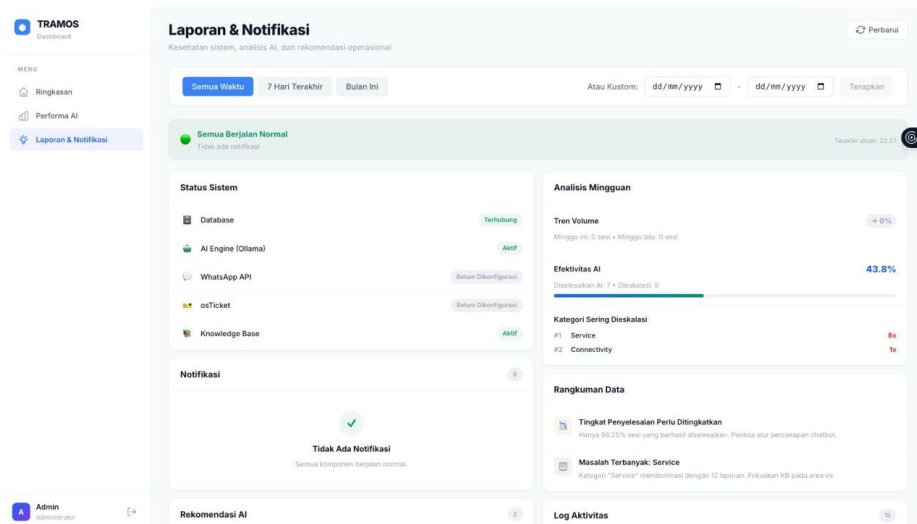


Figure 9 Reports and Notifications Page

3. WhatsApp AI Chatbot Interface

The WhatsApp AI Chatbot acts as the main medium between the drivers/field users and TRAMOS Support System. Using the WhatsApp app itself, drivers are able to report their issues directly using the chatbot without having to download any separate software. WhatsApp AI chatbot uses the WhatsApp Cloud API, Gemini AI processing, PostgreSQL Database Storage, and osTicket as its backend components for automated issue management.

During the first interaction, the chatbot introduces itself, determines the sender's identity, and asks the user about their basic details along with the description of the issue that needs to be resolved. Some pre-defined categories for the issue are also suggested by the system. In case, if the issue description is not clear enough, the system prompts users to provide more information on the issue.

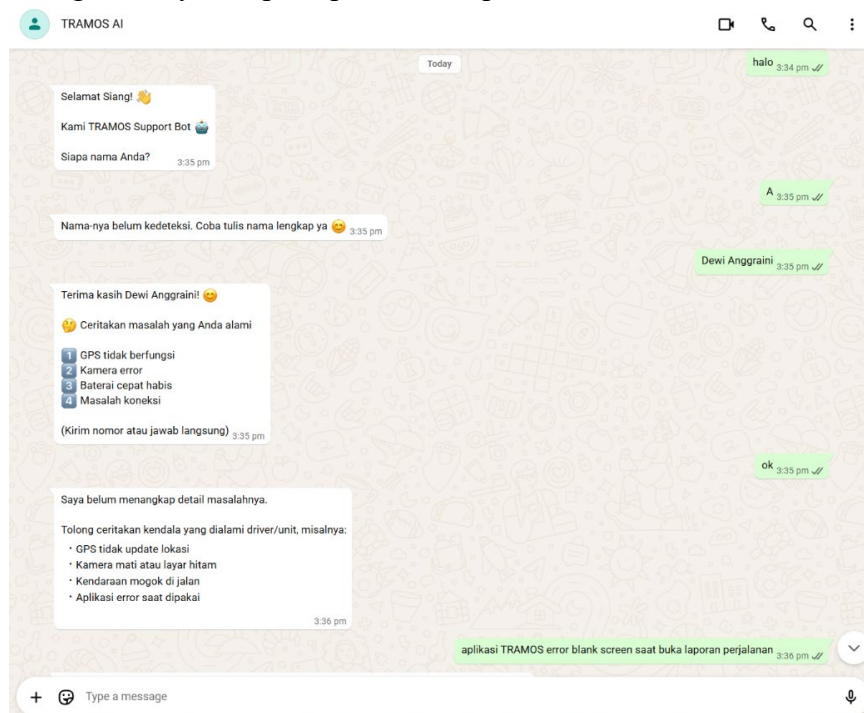


Figure 10 WhatsApp Chatbot Interaction

Following that, the system will analyze the issue type with AI-driven issue analysis and generate auto-fixing suggestions depending on the discovered issue category. In this case, the chatbot accurately detects the issue to be an application issue and provides the users with troubleshooting suggestions accordingly. This indicates the ability of the solution to leverage the conversational AI engine and its internal knowledge management system.

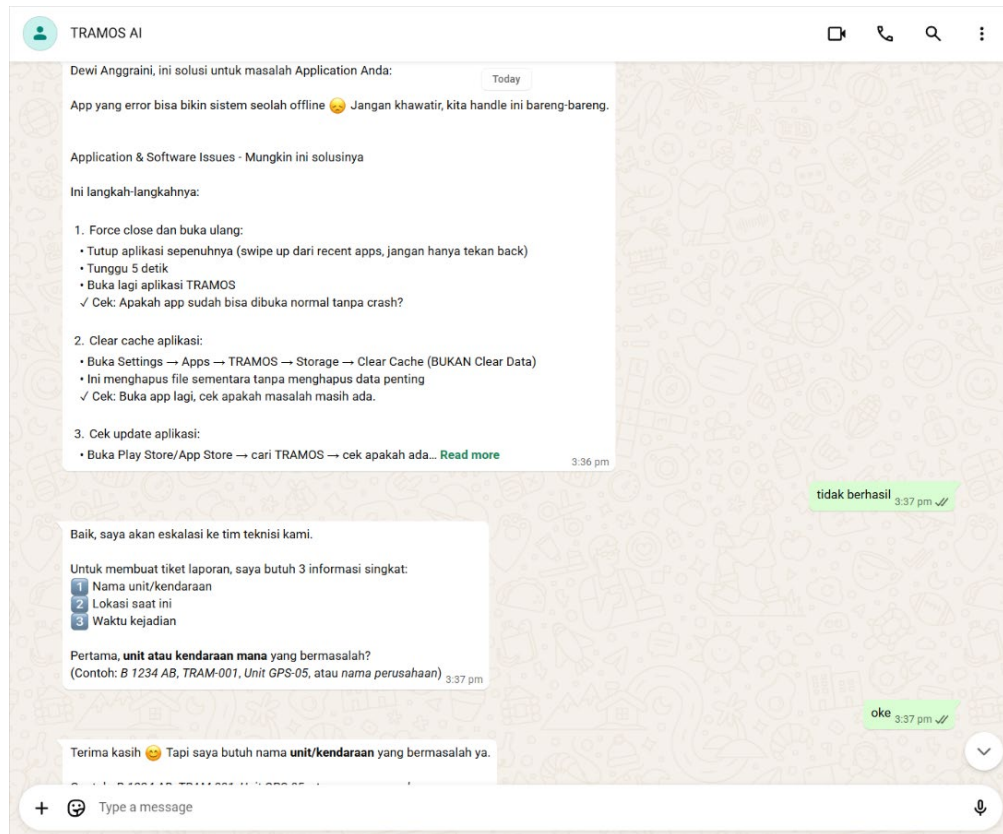


Figure 11 AI Troubleshooting and Escalation Process

In cases where the issue cannot be fixed by means of auto-fixing the system, the chatbot escalates the issue using the structured operational data needed for ticket creation. To escalate the ticket, the system requests information from the user like vehicle/unit identifier, current location, and the time at which the issue occurred. Validation occurs after each request of information to ensure data consistency for better support.

TRAMOS AI

1 Ya, lanjutkan buat tiket
2 Tidak, saya mau perbaiki 3:38 pm

tidak, data salah 3:39 pm ✓

Baik, kita ulangi datanya ya.

Unit atau kendaraan mana yang bermasalah?
(Contoh: B 1234 AB, TRAM-001, Unit GPS-05, atau nama perusahaan) 3:39 pm

Unit: Unit GPS-06 3:39 pm ✓

Selanjutnya, di mana lokasi unit tersebut saat ini?

Bisa sebut:

- Nama kota/kabupaten (misal: Jakarta, Surabaya)
- Area/jalan (misal: Tol Cikampek KM 5, Bandara Soetta)
- Pool/garasi (misal: Pool Cakung, Depo Cilincing)

3:39 pm

Lokasi: Depo Cilincing

Terakhir, kapan masalah ini terjadi?

Bisa jawab dengan:

- Jam spesifik (misal: 14:30, jam 3 sore)
- Waktu umum (misal: tadi pagi, kemarin sore)
- sekarang / barusan jika baru terjadi

3:40 pm

Depo Cilincing 3:40 pm ✓

14.30 3:41 pm ✓

Ringkasan Tiket Laporan

+ Type a message

Figure 12 Ticket Data Collection Workflow

Once all information is provided and collected, the chatbot summarizes the ticket and prompts the user to confirm. Upon confirmation, the chatbot creates the ticket automatically using the osTicket API integration and presents the user with the generated ticket ID. This allows any unresolved operational issue to get escalated to the tech support department while ensuring conversation traceability throughout the whole process from WhatsApp chatbot, database, analytics dashboard to ticketing software.

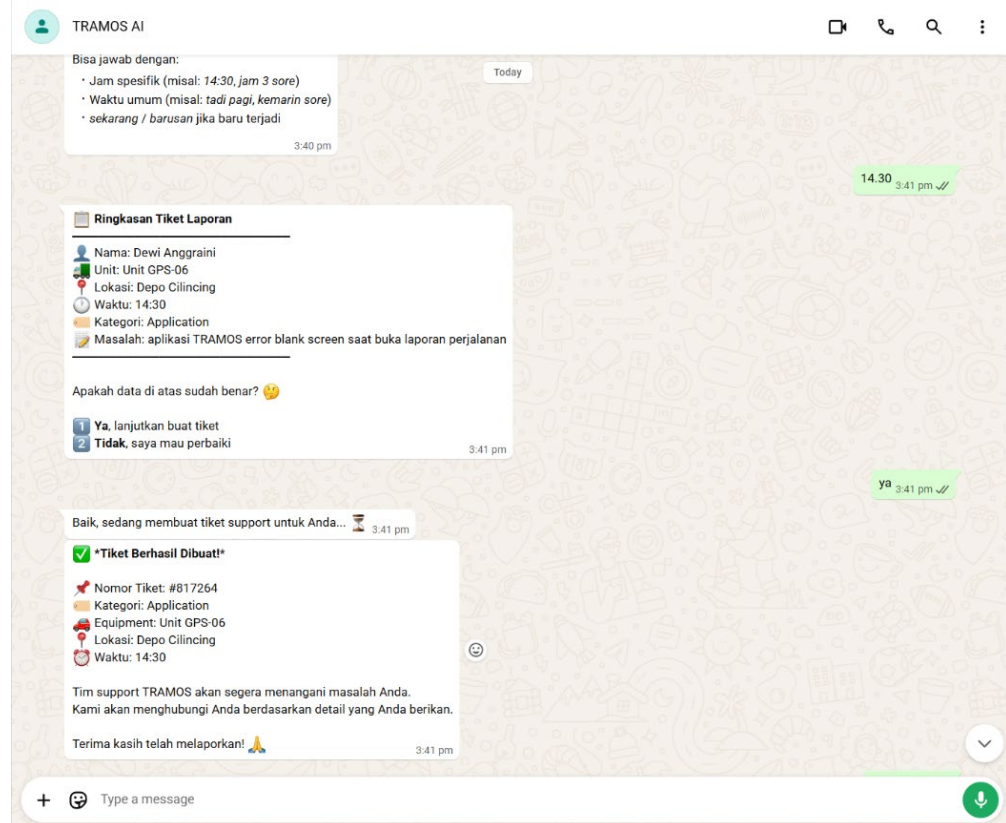


Figure 13 Automatic Ticket Creation Confirmation

4. Email Notification Integration

The TRAMOS solution also integrates an auto-email notification feature once the ticket has been successfully filed. Once the chatbot is done processing the ticket creation process using the osTicket API, the TRAMOS system sends an email notification containing all the details related to the filed ticket to the designated support email address.

The email notification will include relevant data such as ticket ID, ticket category, user details, issue description, ticket priority level, date/time stamp, and summary of the AI conversation. Further, the email notification includes a direct shortcut button to view the specific ticket in the osTicket management system. This way, the ticketing operator does not need to search the ticket from the list; simply clicking on the provided link allows quick access to the ticket.

This kind of integration ensures that support ticketing becomes more responsive and accessible to operators in the management system.

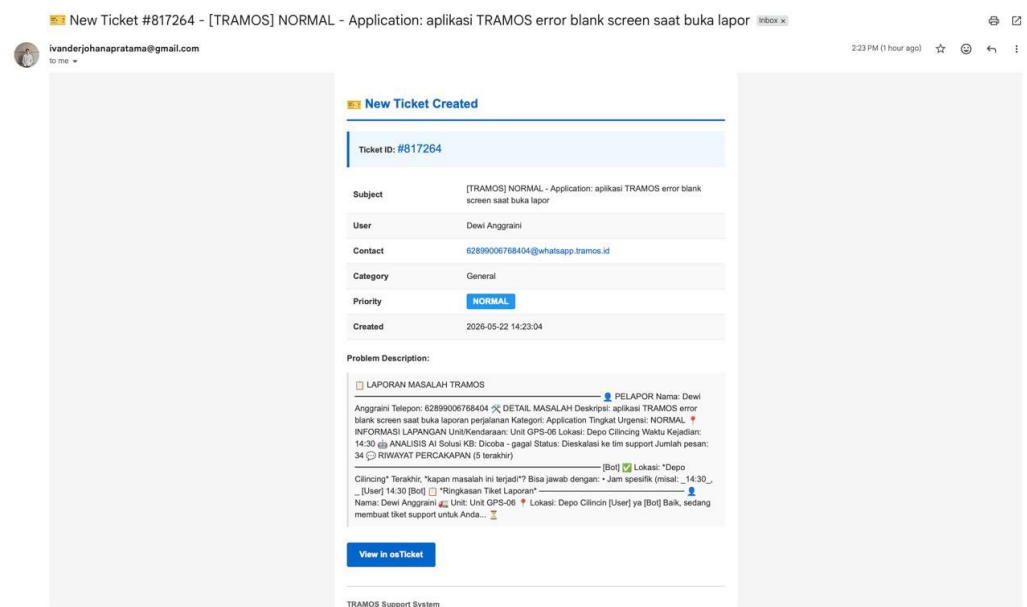


Figure 14 Automatic Email Notification After Ticket Creation

5. osTicket Ticket Management Interface

Once the ticket is created using the TRAMOS chatbot application, all escalation information will be seamlessly synchronized into the osTicket helpdesk application through the use of REST API. The osTicket application serves as the operational management center where technical issues raised through the TRAMOS chatbot application are monitored, managed, and handled by the support operators.

The ticket details page provides a full operational view including ticket status, priority level, user contact, issue category, location of incident, vehicle/unit details, time of occurrence, and AI analysis. The integration also retains conversation context collected while handling issues on the chatbot application allowing the support team to understand the nature of the problem without having to troubleshoot it again.

The ticket management application offers various tools for managing operational activities such as SLA compliance tracking, ticket assignment, ticket status tracking, and ticket thread management among others. This integration helps escalate any unresolved issues originating from the AI chatbot solution into an organized support management environment.

The screenshot displays the osTicket web interface. At the top, there's a navigation bar with tabs for Dashboard, Users, Tasks, Tickets (selected), and Knowledgebase. Below this is a sub-navigation bar with links for Open, My Tickets, Closed, Search, and New Ticket. The main content area shows the details for Ticket #817264, titled "[TRAMOS] NORMAL - Application: aplikasi TRAMOS error blank screen saat buka lapor". The ticket status is Open, Priority is Normal, Department is Support, and it was created on 05/22/26 at 14:23:04. The user is Test User (10) with email adm.helpdesk.tramos@gmail.com and source API. The ticket is currently unassigned. Below the ticket details, there's a section for the Ticket Thread (1) showing a post from Test User on 05/22/26 at 14:23:04. The post content includes a report of a TRAMOS application error, user details (Dewi Anggraini), problem details (blank screen on report), and AI analysis results (KB search failed, escalated to support).

osTicket Welcome, admin. | [Admin Panel](#) | [Profile](#) | [Log Out](#)

Dashboard Users Tasks **Tickets** Knowledgebase

Open My Tickets Closed Search New Ticket

Ticket #817264

[TRAMOS] NORMAL - Application: aplikasi TRAMOS error blank screen saat buka lapor

Status: Open **User:** Test User (10) (Manage Collaborators)
Priority: Normal **Email:** adm.helpdesk.tramos@gmail.com
Department: Support **Source:** API
Create Date: 05/22/26 14:23:04

Assigned To: — Unassigned — **Help Topic:** None
SLA Plan: Default SLA **Last Message:** 05/22/26 14:23:04
Due Date: 05/27/26 14:23:04 **Last Response:**

Ticket Thread (1) Tasks

Test User posted 05/22/26 14:23:04 [TRAMOS] NORMAL - Application: aplikasi TRAMOS error b...

LAPORAN MASALAH TRAMOS

PELAPOR
Nama: Dewi Anggraini
Telepon: 62899006768404

DETAIL MASALAH
Deskripsi: aplikasi TRAMOS error blank screen saat buka laporan perjalanan
Kategori: Application
Tingkat Urgensi: NORMAL

INFORMASI LAPANGAN
Unit/Kendaraan: Unit GPS-06
Lokasi: Depo Cilincing
Waktu Kejadian: 14:30

ANALISIS AI
Solusi KB: Dicoba - gagal
Status: Dieskalasi ke tim support

Figure 15 osTicket Ticket Detail Management Interface

C. COMPONENT COST ANALYSIS

In this section, an estimate for the cost incurred in supporting the development of the TRAMOS system is provided. The cost analysis involves software packages, AI services, communication facilities, and infrastructure elements involved during the course of development. Most of the development tools used in this development initiative are open source, thus resulting in low operational costs incurred in developing the system.

No.	Item	Unit	Price per Unit	Total
1.	osTicket Helpdesk Software	1 installation	Rp. 0	Rp. 0
2.	ngrok Free Plan Development Tunnel	1 account	Rp. 0	Rp. 0
3.	WhatsApp Cloud API Development Testing (free trial)	1 account	Rp. 0	Rp. 0
4.	Google Gemini API	/1M tokens	Rp. 45.000	Rp. 45.000
5.	WhatsApp Number for AI Chatbot	1 number	Rp. 15.000	Rp. 15.000
6.	Monthly Phone Credit (Pulsa)	1 month	Rp. 5.000	Rp. 5.000
TOTAL				Rp. 65.000

D. FUNCTIONAL TESTING

Functional testing in the TRAMOS system is conducted by simulating real user interactions across all implemented modules, including authentication, WhatsApp chatbot processing, ticket escalation, analytics dashboards. This type of testing is performed through independent evaluation of each of the modules through different test inputs to ensure that the application is behaving in accordance with the required functional requirements. The functional testing results provided below were conducted based on the implementation of the following modules:

1. Chatbot Dialog Flow Testing

No.	Scenario	Every Possible Input	Expected Output	Output Result
1.	New conversation started	First WhatsApp message	New session created	Session created successfully
2.	Greeting interaction	“halo”, “selamat pagi”	Bot asks user name	Greeting flow successful
3.	Valid name input	“Ivander Johana”	Name stored and next step displayed	Name stored successfully
4.	Invalid name input	Empty, numeric-only, short input	Bot requests valid name	Validation successful
5.	Valid issue description	“GPS tidak update”	Category analyzed	Issue analyzed successfully
6.	Short issue description	Empty or unclear message	Bot asks for more details	Validation successful
7.	Knowledge base solution found	Known issue category	AI troubleshooting displayed	Solution displayed successfully
8.	Knowledge base solution unavailable	Unknown issue category	Escalation flow started	Escalation started successfully
9.	Solution successful	“ya”, “berhasil”	Session resolved	Resolution successful
10.	Solution failed	“tidak”, “masih error”	Ticket escalation flow started	Escalation successful
11.	Session timeout	No user activity	Session expired automatically	Timeout handled successfully

2. Ticket Creation Testing

No.	Scenario	Every Possible Input	Expected Output	Output Result
1.	Bot asks for vehicle unit	User issue cannot be resolved by AI	Bot requests vehicle/unit information from user	Bot asks vehicle successfully

2.	User input valid vehicle unit	“TRAM-001”, “B 7821”, “Unit 5”	Vehicle unit information stored by system	Vehicle unit stored successfully
3.	User inputs invalid vehicle unit	Empty text, “ok”, unrelated input	Bot asks user to re-enter valid vehicle unit	Validation handled successfully
4.	Bot asks for issue location	Vehicle unit already provided	Bot requests issue location information	Bot asks location successfully
5.	User input valid location	“Jakarta”, “Tol Cikampek KM 45”	Location information stored by system	Location stored successfully
6.	User input invalid location	Empty or unclear location	Bot asks user to re-enter valid location	Validation handled successfully
7.	Bot asks for issue time	Vehicle unit and location already provided	Bot requests issue occurrence time	Bot asks issue time successfully
8.	User input issue time	“14.30”, “jam 3 sore”, “tadi pagi”	Issue time stored by system	Time information stored successfully
9.	Bot displays confirmation summary	Vehicle unit, location and time completed	Bot displays collected issue summary or confirmation	Confirmation summary displayed successfully
10.	User confirm ticket creation	“ya”, “confirm”, “benar”	System creates ticket automatically	Ticket created successfully
11.	User rejects ticket confirmation	“tidak”, “ubah”	Bot restarts data collection process	Flow restarted successfully
12.	System connects to osTicket API	Valid API key and base URL	Ticket synchronized into osTicket dashboard	Synchronization successfully
13.	Bot sends ticket number to user	Ticket successfully created	User receives generated ticket number	Ticket notification sent successfully
14.	System stores ticket data into database	Ticket successfully created	Ticket, analytics, and resolution records stored	Database updated successfully

3. Authentication and Analytics Dashboard Testing

No.	Scenario	Every Possible Input	Expected Output	Output Result
1.	User registers new account	Name, email, password	New account created and OTP generated	Registration successful

2.	User registers using existing verified email	Existing verified email	Registration rejected and error displayed	Error handled successfully
3.	User registers using existing unverified email	Existing unverified email	System resends OTP verification	OTP resent successfully
4.	User input correct OTP	Valid OTP verification code	Account verified successfully	Verification successful
5.	User input incorrect OTP	Invalid OTP code	Verification rejected and error displayed	Error handled successfully
6.	User inputs expired OTP	Expired OTP code	OTP rejected by system	Expired OTP handled successfully
7.	User requests OTP resend	Valid unverified account	New OTP generated and sent	OTP resent successfully
8.	User logs in with valid credentials	Correct email and password	JWT token generated and user redirected to dashboard	Login successfully
9.	User logs in with unverified account	Correct credentials but account not verified	Verification warning displayed	Restriction handled successfully
10.	User logs in with incorrect password	Invalid password	Login rejected and error displayed	Error handled successfully
11.	User logs in using Google authentication	Valid Goggle account	User authenticated and redirected to dashboard	Google login successful
12.	System verifies JWT token	Valid authentication token	User session validated	Token verification successful
13.	User accesses analytics dashboard	Valid authentication session	Dashboard overview displayed	Dashboard loaded successfully
14.	User filters dashboard analytics	Selected start and end dates	Filtered charts and KPI data displayed	Filtering successful
15.	User logs out from system	Logout request	Session terminated and redirected to login page	Logout successful

E. MANUAL GUIDE

1. SYSTEM BUILD DOCUMENTATION FROM THE SOURCE (Developer Perspective)

1) Prerequisites

- Python 3.10+
- PostgreSQL
- Node.js 16+
- Npm
- Ngrok
- osTicket
- Meta WhatsApp Business Account
- Gemini API Key

2) Backend installation

```
cd /Users/vdr/Documents/TRAMOS
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

3) Database Setup

Create PostgreSQL database:
`createdb tramos_db`

4) Run backend server

```
uvicorn main:app --reload --host 0.0.0.0 --port 9999
```

5) Frontend installation

```
cd /Users/vdr/Documents/TRAMOS/dashboard
npm install
npm run dev
```

6) Ngrok Setup

```
ngrok start --config ngrok.yml
```

7) osTicket integration

- Configure API Key in osTicket
- Add osTicket Base URL into .env
- Test Ticket API Connection

8) WhatsApp Webhook Setup

- Open Meta Developer Dashboard
- Configure Webhook Callback URL
- Verify Token
- Subscribe to Message Events

2. END-USER SYSTEM INSTALLATION

(User Perspective)

1) Driver/Field User Requirements

- Smartphone (Android/iOS)
- WhatsApp Application
- Internet Connection

Steps:

- Save TRAMOS WhatsApp number
- Open WhatsApp
- Send issue report message
- Follow chatbot instructions

2) Dashboard User Requirements

- Laptop/Desktop Browser
- Internet Connection
- Dashboard Account

Steps:

- Open dashboard URL
- Login using email/password or Google
- Access analytics dashboard

3) Support Operator Requirements

- osTicket Account
- Browser
- Internet Connection

Steps:

- Open osTicket dashboard
- Login using operator account
- Monitor and manage incoming tickets

3. USER GUIDE PER USER ROLE

This section highlights the manner in which the interaction between the user roles and the TRAMOS application takes place based on their respective responsibilities and permissions within the system. Each role has been formulated to facilitate the efficient functioning of TRAMOS.

User Role	Page	Details
Driver / Staff	WhatsApp Chatbot	Send issue report, receive AI solution, create support ticket
Management Analyst	Analytics Dashboard	View analytics summary, category charts, ticket statistics and operational insights
Support Operator	osTicket Dashboard	Review ticket, update ticket status, resolve issues

REFERENCES

- Al-Hawari, F., & Barham, H. (2019). A machine learning-based help desk system for IT service management. *International Journal of Advanced Computer Science and Applications*, 10(4), 1–12.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., et al. (2001). *Manifesto for agile software development*. The Agile Alliance.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv*.
<https://doi.org/10.48550/arXiv.1810.04805>
- Docker Inc. (2024). *Docker documentation*. <https://docs.docker.com>
- Erl, T. (2016). *Service-oriented architecture: Concepts, technology, and design*. Pearson Education.
- FastAPI. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Google. (2024). *Gemini API documentation*. <https://ai.google.dev>
- Gupta, A., & Sharma, R. (2022). Predictive analytics for issue prevention in logistics platforms. *IEEE Transactions on Intelligent Transportation Systems*, 23(8), 11234–11245.
- IEEE. (2024). *IEEE software requirements specification standards*. IEEE Xplore.
- Jetir. (2025). Online chatbot-based ticketing system: AI-powered helpdesk solution for automated ticket generation. *Journal of Emerging Technologies and Innovative Research*.
- JSON Web Token. (2024). *JWT introduction and specifications*. <https://jwt.io>
- Laudon, K. C., & Laudon, J. P. (2023). *Management information systems: Managing the digital firm* (17th ed.). Pearson Education.
- Mauliana, P., Wiguna, W., & Permana, A. Y. (2020). Pengembangan e-helpdesk support system berbasis web. *Jurnal Responsif*, 2(1), 19–29.
- Meta Platforms. (2024). *WhatsApp Business Platform API documentation*.
<https://developers.facebook.com/docs/whatsapp>
- ngrok Inc. (2024). *ngrok documentation*. <https://ngrok.com/docs>
- osTicket Project. (2024). *osTicket documentation*. <https://docs.osticket.com>
- Papazoglou, M. P., & van den Heuvel, W. J. (2007). Service-oriented architectures: Approaches, technologies and research issues. *The VLDB Journal*, 16(3), 389–415.
<https://doi.org/10.1007/s00778-007-0044-3>
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*.
<https://www.postgresql.org/docs>
- Pressman, R. S., & Maxim, B. R. (2014). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.
- Rachman, F., Setiawan, F. A., & Harimas, A. (2025). Chatbot implementation for digital reporting systems using WhatsApp. In *Proceedings of the Open Society Conference (OSC)*.
- Rao, S., Patel, V., & Kumar, A. (2023). Scalability issues in centralized manual communication systems for service industries. *Computers & Industrial Engineering*, 175, 108854.
- React Team. (2024). *React documentation*. <https://react.dev>

- Siahaan, M., & Kelvin. (2023). Pengembangan helpdesk ticketing system berbasis website dengan menggunakan metode SDLC, XP, dan Scrum. *KLIK: Kajian Ilmiah Informatika dan Komputer*, 4(3), 1399–1410.
- SQLAlchemy Authors. (2024). *SQLAlchemy documentation*. <https://docs.sqlalchemy.org>
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson Education.
- Truss, M., & Boehm, S. (2024). AI-based classification of customer support tickets: State of the art and implementation with AutoML. *arXiv*.
- Uvicorn Contributors. (2024). *Uvicorn documentation*. <https://www.uvicorn.org>
- Vite Team. (2024). *Vite documentation*. <https://vitejs.dev>
- Wijaya, A. S. (2024). Machine learning approaches for helpdesk ticketing systems: A review. *Journal of Theoretical and Applied Information Technology*, 102(5).